

OmOS

Interactive Developer Portfolio Operating System

MASTER PROJECT BLUEPRINT

Complete Build Plan — Phases, Modules & Feature Specifications

Prepared for: Om Ganesh — Full Stack Developer, KLE Technological University

Document type: Technical & Product Blueprint (Planning Reference)

Stack: React · TypeScript · Tailwind CSS · xterm.js · Zustand · Node.js · Supabase · Gemini API

"A terminal that proves the skills, instead of just listing them."

Table of Contents

1. Executive Summary
2. Vision & Problem Statement
3. System Architecture
4. Technology Stack
5. Project Directory Structure
6. Development Roadmap (Phase-by-Phase)
7. Feature Modules — Detailed Specifications (16 Modules)
8. Recommended MVP Scope
9. Full Feature Backlog — Category Overview
10. Engineering Principles
11. Future / Advanced Ideas (Post-v2.0)
12. Risks, Guardrails & Build Recommendations
13. Closing Summary

1. Executive Summary

OmOS is a browser-based operating system simulation that functions as an interactive developer portfolio for Om Ganesh, a Full Stack Developer. Instead of a traditional scrolling website, visitors boot into a simulated OS and explore projects, skills, resume, GitHub activity, and architecture diagrams through a real, working Linux-like terminal. The project is simultaneously the **portfolio** and the **proof** — every command a recruiter runs is itself a demonstration of frontend engineering, state management, systems thinking, and UX design.

This document consolidates the full research and planning work into a single build reference. It covers system architecture, the complete technology stack, the project's folder structure, a phase-by-phase development roadmap with concrete deliverables, detailed specifications for every planned module, the recommended MVP cut, the full long-tail feature backlog, and a set of engineering guardrails and risk notes to keep the build realistic and shippable.

The guiding principle throughout is simple: **every feature must either help a recruiter understand the developer faster, or directly demonstrate an engineering skill**. Anything that does neither is explicitly deprioritized, regardless of how technically interesting it is to build.

2. Vision & Problem Statement

Traditional portfolio websites are passive. A recruiter scans a page for two to five minutes, skims a project list, and moves on — often missing the work that would have mattered most to them. OmOS reframes the portfolio as an **experience** rather than a document:

- **Creates engagement** — a real terminal invites exploration instead of passive scrolling.
- **Demonstrates skill directly** — the portfolio's own construction (parser, virtual filesystem, state management, API integration) is the proof of ability, not just a claim on a resume line.
- **Guides discovery** — Recruiter Mode and the AI assistant actively surface the most relevant content instead of hoping the visitor finds it themselves.
- **Is memorable** — a unique, well-executed concept stands out sharply from templated portfolio sites, which is disproportionately valuable in a saturated job market.

Vision statement: Build a memorable flagship portfolio that feels like a real operating system, while remaining ruthlessly focused on helping a recruiter understand the developer in under five minutes. Novelty is the hook; clarity and speed-to-signal are the actual product.

3. System Architecture

OmOS is architected in layered fashion, mirroring a real operating system's boot-to-application flow. Each layer has a single responsibility and communicates with adjacent layers through a well-defined interface, which keeps modules independently testable and replaceable.

3.1 Layer Flow

Window Manager

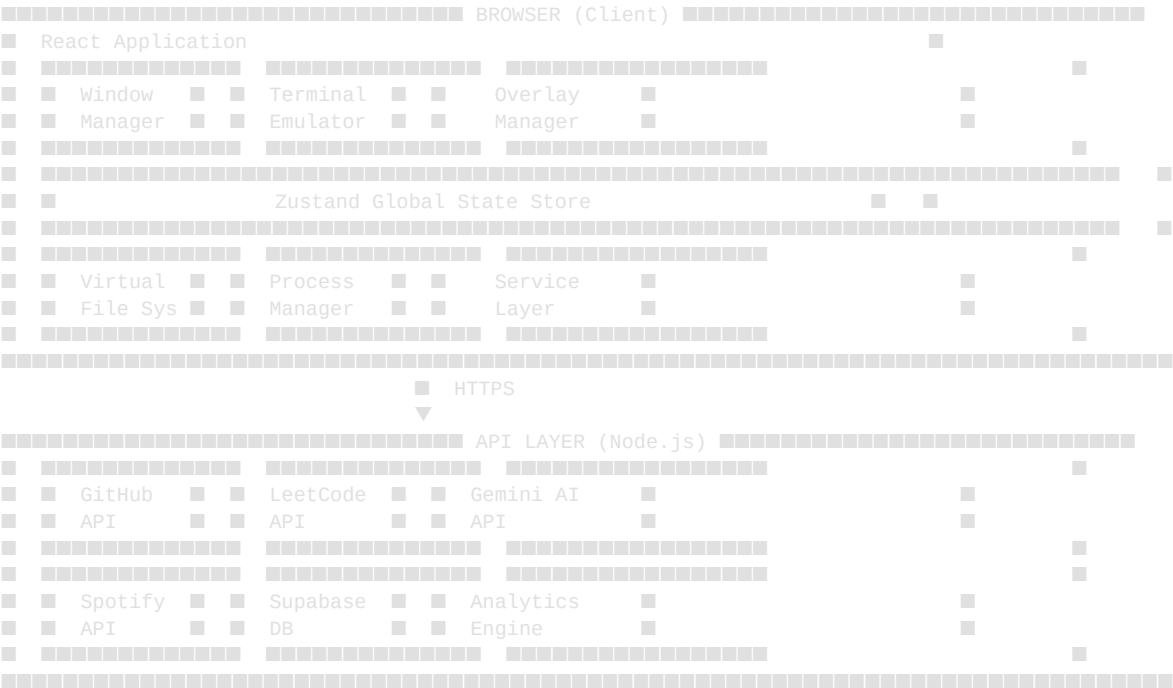




3.2 Client / Server Split

The **client** (React application) owns the entire OS simulation — window manager, terminal, shell parser, virtual filesystem, process manager, and Zustand state stores all run fully in-browser. The **server** (Node.js/Express, or Supabase Edge Functions) exists for exactly one reason: to proxy calls that require a secret API key or persistent storage — GitHub, LeetCode, Gemini AI, and analytics writes. No portfolio content or simulation logic lives on the server; it is a thin, stateless secret-holder and data-relay.

3.3 High-Level Diagram



4. Technology Stack

The stack favors mature, well-documented libraries over exotic tooling — the goal is to spend engineering time on the OS simulation itself, not on fighting the build tools around it.

Layer	Technology	Purpose
Frontend Framework	React 18 + TypeScript	UI components, rendering, type safety
Terminal Emulator	xterm.js	Full terminal rendering & keystroke capture
Styling	Tailwind CSS + Framer Motion	Utility-first styling and smooth animation
State Management	Zustand	Global OS state (filesystem, shell, windows, theme)
Routing	React Router v6	URL-based navigation & deep-linking
Icons	Lucide React	Consistent iconography
HTTP Client	Axios	API communication
Markdown Rendering	React Markdown	Render .md files inside the virtual filesystem
Backend Runtime	Node.js + Express (or Supabase Edge Functions)	Secret-holding proxy for external APIs
Database	Supabase (PostgreSQL)	Persistent data: analytics, achievements, SQL-CLI demo data
AI Integration	Google Gemini API (2.0 Flash)	Portfolio-scoped AI assistant
Real-time	Socket.io	Multiplayer / visitor-presence features (advanced phase only)
Deployment	Vercel (frontend) + Railway/Render (backend)	Hosting and CI/CD
Monitoring	Sentry	Error tracking in production

5. Project Directory Structure

The codebase is split into a **client** app (the entire OS simulation) and a thin **server** (secret proxy + persistence), with shared documentation and tooling at the root. This separation means the client can be developed and demoed almost entirely without the backend running.

```

omos-portfolio/
├── client/                                React frontend (the OS itself)
│   ├── src/
│   │   ├── components/                  terminal/, kernel/, shell/, filesystem/,
│   │   │   └── ── window/, portfolio/, recruiter/, ai/,
│   │   │   └── ── themes/, games/, common/
│   │   ├── kernel/                     boot/, filesystem/, process/, service/, network/
│   │   ├── shell/                      commands/{builtins,portfolio,external,advanced,fun}/,
│   │   │   └── ── parser/, history/, autocomplete/, execution/
│   │   ├── services/                  api/, github/, ai/, analytics/, storage/
│   │   ├── stores/                   osStore, terminalStore, filesystemStore,
│   │   │   └── ── windowStore, themeStore, processStore, settingsStore
│   │   ├── hooks/                     useTerminal, useKeyPress, useCommand, useFileSystem, useTheme
│   │   ├── utils/                     helpers/, formatters/, constants/
│   │   ├── types/                     system, filesystem, command, process, global
│   │   └── themes/                    20 built-in theme definitions
│   └── package.json, tsconfig.json, vite.config.ts, tailwind.config.js
├── server/                              Node.js backend (thin proxy + persistence)
│   ├── src/
│   │   ├── routes/                    github, leetcode, ai, analytics, portfolio
│   │   ├── controllers/, services/, models/, middleware/ (auth, cors, rateLimit)
│   │   └── utils/                     logger, validation, encryption
│   └── package.json, tsconfig.json, .env.example
├── docs/                                API.md, ROADMAP.md, CHANGELOG.md
├── ── architecture/{system-design, database-schema, deployment}.md
├── tests/                              unit/, integration/, e2e/
├── scripts/                            deploy.sh, seedDatabase.ts, generateAssets.ts
└── docker-compose.yml, Dockerfile, vercel.json, README.md, LICENSE

```

Note on documentation scope: a CONTRIBUTING.md is only worth maintaining if external contributors are genuinely expected on a solo portfolio project; for v1.0 it can be folded into the README as a single "how this is built" section to save maintenance overhead.

6. Development Roadmap (Phase-by-Phase)

The build follows five phases, each ending in a working, demoable state rather than a pile of half-finished features. Every phase has an explicit **"done-when"** condition and an explicit **"excluded"** list, so scope discipline is written down rather than assumed.

Version	Focus	Target Duration	Done-When
v0.1	Foundation	Week 1–2	Project scaffolding, state stores, and a real typing terminal exist.
v0.5	Working Shell	Week 3–6	Boot sequence, virtual filesystem, and 15+ commands work end-to-end.
v1.0	Portfolio-Ready	Week 7–9	Projects, resume, skills, GitHub integration, and 2-3 themes are live and demoable.
v1.5	Recruiter Edition	Week 10–11	Guided tour, quick-stats, and role-based filtering are functional.
v2.0	AI Edition	Week 12–14	Portfolio-scoped AI assistant is live with rate limiting and guardrails.
v3.0	Advanced OmOS	Ongoing / optional	Plugins, package manager, networking sim, games, multiplayer — only after v1.0–v2.0

6.1 Phase 0 — Foundation (Week 1–2)

Goal: establish project infrastructure and a real, typing terminal before any OS logic is built.

0.1 Project Setup (Day 1–3)

- Initialize the repo with Vite + React + TypeScript.
- Configure ESLint, Prettier, and the folder structure described in Section 5.
- Set up environment variables, a basic GitHub Actions CI/CD pipeline, and Docker config.

0.2 State Management Setup (Day 4–5)

Implement the core Zustand stores: **osStore** (boot state, uptime, user identity), **terminalStore**, **filesystemStore**, **windowStore**, **themeStore**, and **settingsStore**. These stores are the single source of truth the rest of the app reads from — getting their shape right early avoids painful refactors later.

0.3 Terminal Foundation (Day 6–10)

Integrate xterm.js with the FitAddon (responsive resizing) and WebLinksAddon (clickable URLs in output). Implement raw keystroke handling (Enter, Backspace, printable characters) and a basic prompt. This phase deliberately excludes command parsing — the goal is only to prove that typed input reaches the app and rendered output reaches the screen.

Deliverables: a running terminal that accepts typed input, echoes it, and shows a prompt — with unit tests confirming input handling.

6.2 Phase 1 — Core OS (Week 3–6)

1.1 Boot Sequence (Week 3)

A 5–8 second boot animation across four phases — BIOS (1s), memory test (2s, animated progress), kernel initialization (2s, service-loading log lines), and portfolio data loading (2–3s). A skip-on-keypress option is mandatory so repeat visitors and impatient recruiters aren't forced to wait. A Konami-code easter egg during boot can unlock a special theme.

1.2 Virtual File System (Week 4–5)

An inode-based simulated Linux filesystem (root, home, etc, usr, var, proc, tmp) with real directories for Projects, Experience, Education, Skills, Certificates, Achievements, Contact, and a hidden `.hidden/` directory for easter-egg content (failed projects, funny bugs, first website). Each inode tracks type, permissions, owner, size, timestamps, and content — file types (`.md`, `.pdf`, `.sh`, `.html`, `.json`, `.txt`, images) each get an appropriate renderer.

1.3 Shell Parser & Command Execution (Week 5–6)

A proper tokenizer that handles quoted strings and flags, feeding a command registry pattern (name, aliases, argument schema, handler, help text) rather than a hardcoded if/else chain. This is the single most important architectural decision in the whole project: it lets **help** auto-generate itself from the registry, keeps command additions cheap, and supports pipes (`| grep`, `| sort`, `| wc -l`), redirection (`>`, `>>`), and command chaining (`&&`, `||`, `;`) without special-casing every combination.

Deliverables: boot sequence, full virtual filesystem, and 15+ working commands (`ls`, `cd`, `pwd`, `cat`, `tree`, `history`, `whoami`, `help`, and more) with pipe and redirection support.

6.3 Phase 1.5 — Portfolio-Ready (Week 7–9)

Build the actual portfolio content on top of the working shell: project explorer, resume viewer, skills display, GitHub integration (real API, not mocked data), and 2–3 fully polished themes (not all 20 — polish beats quantity here). This is the first version safe to actually share publicly, since it delivers the core promise: explore my work through a real terminal.

6.4 Phase 2 — Recruiter Edition (Week 10–11)

Add the Recruiter Mode module (Section 7.10): a guided tour with explicit timed steps, role-based project/skill filtering, and quick-stats summaries. This phase exists specifically to serve the 5-minutes-to-understand-the-developer promise from the vision statement — it should be built and user-tested with an actual recruiter or hiring-adjacent friend before moving on.

6.5 Phase 3 — AI Edition (Week 12–14)

Wire in the Gemini-powered AI assistant, scoped strictly to portfolio knowledge (Section 7.9). This phase must ship with rate limiting and prompt-injection safeguards from day one — a public "ask" command connected to a real LLM is an obvious target for cost-drain abuse, and retrofitting safety after launch is much riskier than building it in from the start.

6.6 Phase 4 (Optional) — Advanced OmOS (v3.0)

Plugin system, package manager simulation, SQL CLI, network stack simulation, built-in games, code playground, and multiplayer/presence features. These are genuinely fun engineering exercises, but none of them

are required for the core recruiter-facing promise. **Recommendation:** only start this phase after v1.0–v2.0 have been shown to real recruiters and validated as impressive on their own — otherwise it's very easy to spend months here instead of shipping.

7. Feature Modules — Detailed Specifications

Sixteen modules make up the complete OmOS vision. Each is described below with its purpose, its core capabilities, and — critically — what it is meant to prove to a recruiter or demonstrate engineering-wise. Modules are ordered roughly by build priority.

7.1 Boot Sequence Module

Purpose: Creates an authentic OS boot experience on page load.

- Four timed phases: BIOS → memory test → kernel init → portfolio load (5–8s total)
- Skip-on-keypress so repeat visitors aren't stalled
- Konami-code easter egg unlocking a special theme
- Boot log rendered as scrolling monospace text with a blinking cursor

Why it matters: Demonstrates animation sequencing, async/await orchestration, and attention to first-impression UX.

7.2 Virtual File System Module

Purpose: Simulates a real Linux filesystem structure to house all portfolio content.

- Inode-based model: type, permissions, owner, size, timestamps, content, metadata
- Realistic tree: /home/om/{Projects, Experience, Education, Skills, Certificates, Achievements, Contact, .hidden}, plus /etc, /usr, /var, /proc, /tmp
- Per-file-type rendering: markdown preview, PDF viewer, executable .sh scripts, JSON pretty-printer
- Core operations: cd, pwd, ls, cat, touch, mkdir, cp, mv, rm, find, grep, stat, chmod

Why it matters: Proves data-structure design skill (tree/graph modeling) and is the foundation every other command depends on.

7.3 Terminal Emulator & Shell Module

Purpose: The real, typing Linux-like terminal that is the entire interface.

- xterm.js rendering with command history (↑↓ navigation) and tab completion
- Tokenizer + AST-based parser supporting pipes, redirects (>, >>), and chaining (&&, ||, ;)
- Command registry pattern (name, aliases, arg schema, handler, help text) instead of hardcoded branching
- Configurable terminal settings: font, cursor style, scrollbar, opacity

Why it matters: The single highest-leverage engineering artifact in the project — a real parser is a genuine systems-programming demonstration, not just UI polish.

7.4 Process Manager Module

Purpose: Simulates OS-level process management, doubling as a creative skills visualization.

- Full process model: PID, PPID, state, CPU%, memory, priority, uptime, threads
- ps / top / htop-style views, with htop rendering skills as long-running processes (e.g., Java at 85% CPU with 1500h uptime — years of experience as process runtime)
- Job control: jobs, fg, bg, disown for backgrounded commands

Why it matters: Turns a resume's flat skills list into an interactive, self-explanatory visualization — this is a genuinely clever reuse of a systems concept for storytelling.

7.5 Service / Daemon Manager Module

Purpose: Background services simulation mapped to real integrations.

- systemctl-style start/stop/restart/status for services like github-sync, ai-assistant, analytics, backup
- Service logs viewable per-service
- Failed-service detection and reporting

Why it matters: Demonstrates understanding of service-oriented architecture concepts, reframed as an interactive display rather than dry documentation.

7.6 Project Explorer & Resume Module

Purpose: The actual portfolio content: projects, resume, skills, and experience.

- projects command lists real projects with README, tech stack, and demo assets per project
- resume renders/downloads the actual PDF
- skills displays proficiency and years of experience, backed by real self-assessment
- whoami / neofetch show a mini-bio with ASCII branding

Why it matters: This is the actual payload — every other module exists to make this content more engaging and discoverable, not to replace it.

7.7 GitHub Integration Module

Purpose: Live data from real developer activity, not mocked numbers.

- github-activity command pulls a real contribution graph via the GitHub API
- Cached server-side to respect GitHub's unauthenticated rate limits
- Optionally extends to LeetCode progress via the same pattern

Why it matters: Proves the portfolio reflects real, current activity — recruiters trust live data far more than static claims.

7.8 Network Stack Module (Advanced)

Purpose: A simulated network topology and networking command set.

- ping, traceroute, ifconfig, netstat, curl, wget, ssh — all operating against a simulated topology
- A fictional but realistic topology: localhost → portfolio-gateway → isp-router → cloudflare-dns → github.com

Why it matters: A fun, deep-cut engineering demonstration for technically curious visitors — explicitly a v3.0/advanced feature, not core.

7.9 AI Assistant Module

Purpose: An AI assistant scoped strictly to portfolio knowledge.

- ask / ai for natural-language queries, recommend for suggested next steps, explain for project walkthroughs
- interview: a mock-interview simulation with scoring
- rate-me / improve: AI-generated portfolio critique
- Context includes visitor path, viewed projects, and time spent — for more relevant answers

Why it matters: Demonstrates LLM-integration skill directly. **Must ship with:** a system prompt hard-restricted to supplied portfolio data, canned redirects for off-topic questions, and server-side rate limiting per session/IP before any public launch — this is the single highest-risk module from an abuse standpoint.

7.10 Recruiter Mode Module

Purpose: A purpose-built fast path for hiring managers.

- A ~5-minute guided tour: Education (30s) → Skills (45s) → Top Projects (60s) → Live Demo (90s) → GitHub Activity (30s) → Resume & Contact (30s)
- Role-based filtering of projects and skills (e.g., "backend role" surfaces different highlights than "frontend role")
- Quick-stats summary: years of experience, top skills, GitHub activity, relevant project count

Why it matters: Directly serves the project's core vision statement. **Needs explicit definition before build:** does it filter the existing command registry, or run as a fully separate guided route? Decide this before writing the shell parser, since it affects that design.

7.11 Achievement System Module

Purpose: Gamification layer rewarding exploration.

- Categories: Exploration (common), Skill (rare), Secret (epic), Social (rare), Challenge (legendary)
- XP and level system with a visible progress command
- Hidden achievements for easter eggs (Konami code, .hidden directory, sudo rm -rf / joke response)

Why it matters: Adds replay value and rewards curiosity — but is pure delight, not information delivery; keep it visually secondary to the recruiter-facing content.

7.12 Analytics Engine Module

Purpose: Tracks and visualizes visitor behavior.

- Session tracking: commands run, navigation paths, view durations, interactions
- Reporting: realtime stats, daily/weekly reports, command-usage heatmaps
- ASCII-chart generation for in-terminal visualization of the data itself

Why it matters: Demonstrates full-stack data-pipeline thinking, and doubles as content — a stats command showing the most-typed command by visitors is a nice self-referential touch.

7.13 Theming System Module

Purpose: Customizable visual appearance across the whole OS.

- Structured theme objects covering terminal colors, UI colors, effects (scanlines, glow, particles), and typography
- 20 built-in themes spanning modern, retro, and pop-culture-inspired palettes
- theme --random and user-created custom themes via a builder UI

Why it matters: Ship 2–3 fully polished themes for v1.0 rather than 20 mediocre ones — theme count is a v3.0 nice-to-have, not a launch requirement.

7.14 Game Engine Module (Advanced)

Purpose: Built-in terminal games for engagement.

- Snake, Tetris, 2048, Tic-Tac-Toe, Hangman (tech terms), Typing Test, Quiz, Minesweeper, and more
- A shared Game interface (init/start/pause/resume/end) so new games plug into one engine

Why it matters: Purely a delight/engagement feature with no recruiter-signal value — correctly placed in the v3.0 advanced backlog.

7.15 Code Playground Module (Advanced)

Purpose: An interactive, real in-browser code execution environment.

- Multi-language support: Python via Pyodide/WebAssembly (real execution), JavaScript native, Java/C++ via simulated or sandboxed compilation
- Snippet library, project templates, and linting

Why it matters: A genuinely impressive but heavy feature — real code execution in-browser is non-trivial engineering; scope it only after the core OS is solid.

7.16 Version Control (Git) Simulation Module

Purpose: A simulated Git interface showing the portfolio's own development history.

- Standard commands: init, clone, add, commit, push, pull, branch, checkout, merge, log, diff, blame
- `git log --oneline` replays the portfolio's real feature-by-feature build history as commits

Why it matters: A meta, self-referential feature — the portfolio's own changelog becomes explorable git history, which is a nice demonstration of both Git fluency and creative framing.

8. Recommended MVP Scope

The MVP is the smallest set of modules that fully delivers on the project's vision statement — understandable in under five minutes, memorable, and provably well-engineered. Everything else is explicitly deferred until this set is built, polished, and validated with real recruiters.

- Boot animation (Module 7.1)
- A real, working terminal with history and autocomplete (Module 7.3)
- Full virtual filesystem (Module 7.2)
- Project explorer, resume, and skills (Module 7.6)
- Live GitHub integration (Module 7.7)
- Recruiter Mode with guided tour (Module 7.10)
- An architecture-viewer command showing this very system's own design
- A portfolio-scoped AI assistant with guardrails (Module 7.9)
- 2–3 fully polished themes, not the full theme library (part of Module 7.13)

Explicitly excluded from MVP: process manager, service/daemon manager, network stack, database CLI, package manager, achievement system, analytics dashboard, games, code playground, and Git simulation. These are high-value v1.5–v3.0 additions, not launch requirements.

9. Full Feature Backlog — Category Overview

The complete brainstorm underlying this project spans roughly 300+ individual feature ideas. Rather than list every one, they are grouped here by category so the scope is visible at a glance. Implementation should proceed by phased milestone (Section 6), never by attempting the full backlog at once.

Category	Representative Ideas
System Architecture	Window manager, kernel, VFS, process manager, service layer
Boot Sequence	BIOS screen, memory test animation, Konami code, skip option
Filesystem	Inode model, permissions, hidden directories, per-type file rendering
Shell	Tokenizer, pipes, redirects, chaining, history, autocomplete, aliases
Theming	20 built-in themes, custom theme builder, seasonal/time-based themes
Commands	60+ commands across system, file, portfolio, fun, network, process categories
Portfolio Features	Project explorer, resume viewer, skills display, timeline diffing
APIs	GitHub, LeetCode, Spotify, Gemini AI — all real integrations, no mock data
AI Assistant	Chat, recommendations, project explanations, mock interviews, portfolio rating
Recruiter Mode	Guided tour, role-based filtering, quick stats, exportable reports
Analytics	Session tracking, heatmaps, command-usage stats, ASCII charts

Multimedia	Demo GIFs, embedded screenshots, sound effects, ASCII art
Developer Tools	Code playground, SQL CLI, Git simulation, package manager
Persistence	localStorage for history/theme/achievements, Supabase for shared data
Accessibility	Screen-reader fallback, reduced-motion detection, keyboard-only navigation
PWA	Installable app, offline caching of static portfolio content
Security	Rate limiting, input sanitization, AI prompt-injection guardrails
Networking (sim)	ping/traceroute/netstat against a simulated topology
Database CLI	SQL interface over a real Supabase schema (projects, skills, experience)
Package Management	apt-like install/remove for optional themes, games, and features
Gamification	XP, levels, achievements across five rarity/category tiers
Future Expansion	Plugin system, multiplayer presence, voice control, portfolio time-travel

10. Engineering Principles

- **Prefer real APIs over fake data.** GitHub activity, resume content, and skill data should reflect genuine, current information — recruiters can tell the difference, and it undermines trust if they catch a mock.
- **Every feature must provide recruiter value or demonstrate engineering.** If a feature does neither, it belongs in the future-ideas backlog, not the active roadmap.
- **Build modules independently.** The command-registry pattern and clearly separated stores mean each module (filesystem, process manager, theming, etc.) can be built, tested, and demoed in isolation.
- **Maintain living documentation.** README, ROADMAP, and CHANGELOG should be updated alongside each release, not retrofitted at the end.
- **Use Git feature branches and iterative releases.** Each roadmap phase (v0.1 → v3.0) should correspond to a real, tagged release — this also gives the eventual git log --oneline simulation (Module 7.16) genuine history to replay.

11. Future / Advanced Ideas (Post-v2.0)

These ideas are genuinely worth building — but only after the MVP has been validated with real recruiter feedback, since they add engineering depth without adding recruiter-facing clarity.

- Plugin system allowing new commands to be registered without touching core shell code
- Package manager simulation for installing optional themes, games, or features
- SQL CLI querying a real Supabase-backed schema of projects/skills/experience
- Docker simulation demonstrating containerization concepts interactively
- Full network stack simulation (Module 7.8)
- In-browser code playground with real multi-language execution (Module 7.15)
- Analytics dashboard exposing the data collected by Module 7.12
- Achievement system (Module 7.11) and story-mode narrative walkthroughs
- Voice control for hands-free command entry
- Multiplayer / visitor-presence features (who, wall, chat --room=general)
- Portfolio "time-travel" — browsing earlier versions of the portfolio itself as a demonstration of the Git-history simulation

12. Risks, Guardrails & Build Recommendations

A few structural risks are worth calling out explicitly, since they're easy to miss while deep in the feature list.

12.1 Scope vs. the Five-Minute Promise

The full backlog spans 300+ features across plugins, SQL CLI, Docker simulation, and multiplayer — none of which help a recruiter understand the developer faster. **Hard rule:** nothing beyond v2.0 gets built until v1.0 has been shown to real recruiters and produced a genuinely positive reaction. A recruiter who has to discover ask,

hire, and theme before finding the resume is worse off than one on a plain static site.

12.2 Backend Footprint

Node.js/Express and Supabase are both listed in the stack. If Express is only proxying secret API calls (GitHub, Gemini), a couple of Supabase Edge Functions can likely replace it entirely — removing a whole deployment target. Decide this explicitly in Phase 0 rather than defaulting to running both.

12.3 AI Assistant Guardrails

The AI assistant (Module 7.9) is the highest-risk feature from an abuse standpoint. Before public launch it needs: a system prompt hard-restricted to supplied portfolio context (not open-domain chat), canned redirects for off-topic questions, and server-side rate limiting per session/IP. This belongs in the Phase 3 checklist itself, not as a follow-up patch after launch.

12.4 Recruiter Mode Definition

"Recruiter mode" needs one concrete design decision made early: does it filter the existing command registry (same commands, curated subset), or run as a fully separate guided route with its own UI? This choice affects the shell parser's design in Phase 1, so it should be settled in Phase 0/1 planning, not discovered mid-build.

12.5 Documentation Realism

README/ROADMAP/CHANGELOG/CONTRIBUTING is solid practice, but CONTRIBUTING.md implies external contributors a solo portfolio project won't realistically have. Fold it into the README as a single "how this is built" section to avoid maintaining a doc no one will use.

12.6 Validate Before You Build the Rest

The single highest-leverage action available at any point in this roadmap is sitting a real recruiter or hiring-adjacent friend in front of the current build with zero instructions and watching where they get stuck. Terminal-UI portfolios live or die on **discoverability**, not feature count — and this project already has far more feature ideas than it needs to ship a genuinely great v1.0.

13. Closing Summary

OmOS is an ambitious, well-conceived flagship portfolio project with a clear architectural spine — window manager, terminal, shell parser, kernel, virtual filesystem, process manager, and service layer — and a well-organized, phased path from a typing terminal to a fully AI-guided recruiter experience. The technology choices are mature and appropriate for the scope, and the modular command-registry pattern is the right foundation to build 60+ commands on without the codebase collapsing into an unmaintainable pile of special cases.

The single biggest risk is not technical — it's scope discipline. The project should be built in the order laid out in Section 6, validated against real recruiters at the end of each major phase, and the v3.0 advanced backlog (Section 11) should remain exactly that: a backlog, not a roadmap, until the core promise has been proven to work.

This document consolidates the complete OmOS research and planning discussion into a single build reference. It is intended as a living document — update the roadmap and backlog sections as phases complete and priorities shift.